Slice — User's Manual

Building smaller fonts from variable fonts

Version 0.1.0

Contents

1 What Slice is for

A variable font contains a whole design space. One file can hold every weight from Thin to Black, every width from Condensed to Expanded, and everything in between. That is excellent for the web and awkward almost everywhere else: many design applications still handle variable fonts poorly, printers ask for a single static file, and a foundry shipping a retail package usually wants named instances rather than one large variable font.

Slice takes a variable font and gives you a smaller one. You can:

- **Pin** an axis to a single location, freezing it. Pin every axis and the result is a static font.
- **Restrict** an axis to a narrower range, keeping the font variable over a smaller design space.
- **Edit** the `name` records and the style bits, so the result identifies itself correctly rather than claiming to be the font it was cut from.
- **Remove overlapping contours**, which the original Slice never did, and which matters because support for overlapping outlines in design applications remains poor after more than a decade.

Everything happens in your browser. The font is never uploaded anywhere. You can try it now.

2 Two ways to run it

2.1 In a browser

The current build of `main` is published and ready to use:

<https://felipesanches.github.io/slice-web/app/>

Nothing is uploaded. The page is static files and a WebAssembly module; your font is read by the browser, sliced there, and handed straight back as a download. You can confirm that by loading the page and then going offline before you press Slice.

To run it from a checkout instead:

```
./build.sh
python3 -m http.server --directory dist 8080
```

Then open `http://localhost:8080`. `dist/` is a directory of ordinary static files with no server-side component, so you can copy it to any host that serves `.wasm` with the `application/wasm` content type.

2.2 From the command line

```
cargo build --release -p slice-cli
./target/release/slice info MyFont.ttf
./target/release/slice cut MyFont.ttf Bold.ttf --axis wght=800
```

The command line takes exactly the same axis syntax as the interface, and `info` prints exactly what the three editors would show. Full reference in the *Command line* section.

3 The workflow

1. **Open a font.** Drop a file onto the page, or use the file picker. Slice reads the font's `fvar`, `name`, `OS/2` and `head` tables and fills in the three editors.
2. **Fill in the Axis Editor.** This is the only part you must touch.
3. **Optionally edit names and style bits.**
4. **Optionally tick "Remove overlapping contours".**
5. **Press Slice**, and choose where the result goes.

A font with no `fvar` table is not a variable font and is refused at step 1, with a message saying so. There is nothing Slice could do to such a font but copy it.

4 The Axis Editor

One row per axis. The left column is fixed and shows what the font supports, in the form `min : max [default]`. The right column is yours to fill in.

What you want	What you type	Example
Keep the whole axis	nothing	
Pin the axis to one location	a number	`400`
Restrict the axis to a smaller range	`min:max`	`200:700`

Spaces are ignored, so `200 : 700` and `200:700` are the same. A reversed range is sorted, so `700:200` means `200:700`. Scientific notation and a leading decimal point are both accepted, because they are accepted everywhere else a font tool reads a number: `1e3` is one thousand and `.25` is a quarter.

Pin every axis and you get a static font. Leave even one axis with a range, or blank, and the result is still a variable font over a smaller design space.

4.1 The rule about defaults

A restricted range must contain the axis's original default value.

This is not a limitation Slice invented. It is what OpenType calls Level 3 sub-spacing: the compiler can narrow an axis, but it cannot move where that axis's default sits, because every glyph outline in the font is stored as a set of deltas measured *from* the default. Moving it would mean recomputing every delta in the font against a new origin.

So on an axis running `300 : 1000 [300]`, typing `400:700` is refused:

> `The wght range 400:700 does not include the default axis value (300).`
> `This is currently a requirement.`

Type `300:700` instead. If you genuinely need a font whose default weight is 400, pin the axis at 400 to get a static font, or use `fonttools varLib.instancer`, which implements the harder Level 4 case.

4.2 A range is an intersection; a pin is an assertion

These behave differently on purpose.

If you **restrict** an axis to a range wider than the font has — `0:700` on an axis that starts at 300 — Slice quietly clamps it to `300:700`. You asked to keep everything up to 700, and everything up to 700 is what you get.

If you **pin** an axis outside its range — `2000` on an axis that stops at 1000 — Slice refuses:

> `The wght value 2000 is outside the axis range 300 to 1000 supported by this font.`

A pin is a claim that a specific location exists. Silently substituting the nearest one would hand you a font that is not the weight you asked for, and nothing would say so.

4.3 Things that are refused

You typed	What happens
`abc`	`'abc' is not a valid wght axis value.`
`400,5`	Refused — use a decimal point, not a comma
`2000` (past the axis end)	Refused, with the supported range quoted
`400:700` where the default is 300	Refused, with the default quoted
`300:700[500]`	Refused — that is Level 4, see above
nothing at all, on every axis	`You requested the same design space that is supported in the font...`

That last one is worth explaining. If every cell is blank you have not asked for anything, so there is nothing to do. The exception is overlap removal, which is a real change on its own, so ticking that box lets an otherwise-empty form through.

5 The Name Editor

A font's `name` table is how an operating system knows what to call it and which other fonts belong in the same family. If you pin a variable font at Bold and leave the names alone, the result will still call itself by the family's default style name — and your font menu will show two entries claiming to be the same thing.

Slice exposes nine records:

ID	Name	What it is
1	Family	The family name, as used by applications with a four-style model
2	Subfamily	Regular, Bold, Italic or Bold Italic — only those four
3	Unique	A unique identifier for this exact font

ID	Name	What it is
4	Full	The full human-readable name
6	PostScript	The PostScript name. No spaces, ASCII only
16	Typographic Family	The real family name, when there are more than four styles
17	Typographic Subfamily	The real style name, e.g. `SemiCondensed Light`
21	WWS Family	Family name when styles differ only in weight, width and slope
22	WWS Subfamily	The corresponding style name

IDs 1, 2, 3, 4 and 6 are always written. IDs 16, 17, 21 and 22 are written when you put something in them and **removed from the output when you leave them blank** — those four are optional records, and an empty optional record is not the same as an absent one.

Records are read and written for the Windows platform (platform 3, encoding 1, language 1033) only. Every other `name` record in the font — copyright, licence, designer, version, and any localisation into other languages — is carried through untouched.

> ***If you only ever read one paragraph of this manual**: IDs 16 and 17 are how a family with more than four styles tells the operating system which fonts belong together. Fill them in for anything larger than a Regular/Bold/Italic/Bold-Italic set. The original Slice deletes all four of these records on every save; this version does not.*

6 The Bit Flag Editor

Two fields, six checkboxes. These tell software whether the font is regular, bold or italic. They are read from the font when you open it, so leaving the panel alone leaves the font's own values in place.

`OS/2.fsSelection`

Bit	Name	Meaning
0	ITALIC	Glyphs are slanted. Set together with `head.macStyle` bit 1
5	BOLD	Glyphs are emboldened. Set together with `head.macStyle` bit 0
6	REGULAR	The regular style. Mutually exclusive with ITALIC and BOLD
8	WWS	Family name differs from others only in weight, width and slope

`head.macStyle`

Bit	Name	Meaning
0	BOLD	Should agree with `fsSelection` bit 5
1	ITALIC	Should agree with `fsSelection` bit 0

Slice warns — but does not refuse — when the combination is inconsistent: REGULAR set alongside BOLD or ITALIC, or the two BOLD bits disagreeing with each other. The specifica-

tion requires them to agree, and software in the wild reads sometimes one and sometimes the other, so a disagreement shows up as a font that is bold in one application and not in another.

Bits the editor does not expose — `fsSelection` bits 1–4, 7, 9 and up — are preserved exactly as the font had them.

7 Removing overlapping contours

Tick **Remove overlapping contours** and Slice merges each glyph's overlapping shapes into a single outline describing the same filled region.

Why you would want this: a glyph drawn as a stem overlapping a bowl renders correctly under the non-zero winding rule, which every modern rasterizer implements. But a great deal of software still does not cope — some older PostScript workflows, some cutting and engraving software, some applications' own path importers. Overlap removal produces the same picture described in a way that nothing can misread.

Three things to know:

It needs every axis pinned. Merging contours renumbers a glyph's points, and a variable font's `gvar` deltas are indexed *by* point number, so the two cannot both be true. If any axis is left variable you get:

> `Overlap removal needs a static font: gvar deltas are indexed by point number, and merging contours renumbers the points. Pin every axis, or turn overlap removal off.`

For a CFF2 font the reason has the same shape: the variation data lives in the charstrings' `blend` operators, which a redrawn outline no longer carries.

It drops the hinting. TrueType instructions address points by index, so a hinting program written against the old point numbering would move the wrong points after a merge. `prep`, `fpgm`, `cvt` and every glyph's own instructions are removed, and `maxp`'s instruction limits are zeroed to match. A partly-hinted font is worse than an unhinted one.

It leaves clean glyphs alone. A glyph with nothing to merge is passed through with its original points, not rebuilt. Running a clean outline through a boolean engine costs precision and gains nothing.

Composite glyphs are decomposed only when their components actually overlap each other. A composite whose parts do not touch stays a composite, because decomposing it would multiply the font's size for no benefit.

8 Output formats

The extension you type chooses the container:

Extension	Result
`.ttf`, `.otf`	A plain sfnt. These two are the same container; the extension does not change the outline format
`.woff`	WOFF 1.0, zlib-compressed

Extension	Result
`.woff2`	WOFF 2.0, brotli-compressed — the smallest

Naming the output `.ttf` will not convert a CFF font's cubic outlines into quadratic ones, and naming it `.otf` will not do the reverse. Converting outlines is a lossy redraw, and it is not what choosing a file extension should mean.

WOFF and WOFF2 inputs are read as readily as plain sfnt, so you can open a `.woff2` and save a `.ttf`.

9 Command line

```
slice info <FONT> [--all-names] [--json]
slice cut  <FONT> <OUTPUT> [OPTIONS]
```

`info` prints what the three editors would show. `-all-names` adds every `name` record rather than the nine the editor exposes; `-json` emits the same information for scripts.

`cut` options, all repeatable:

Option	Meaning
`-axis TAG=VALUE`	An axis setting, in the Axis Editor syntax
`-name ID=TEXT`	Set a `name` record
'–fs-selection BIT=on\	off'
'–mac-style BIT=on\	off'
`-remove-overlaps`	Merge overlapping contours; needs every axis pinned

An example that produces a named static instance:

```
slice cut Recursive-VF.ttf RecursiveSans-Bold.ttf \
    --axis MONO=0 --axis CASL=0 --axis wght=700 --axis slnt=0 --axis CRSV=0.5 \
    --name 1='Recursive Sans' \
    --name 2='Bold' \
    --name 4='Recursive Sans Bold' \
    --name 6='RecursiveSans-Bold' \
    --fs-selection 5=on --fs-selection 6=off \
    --mac-style 0=on \
    --remove-overlaps
```

Note the `-fs-selection 6=off`: REGULAR and BOLD are mutually exclusive, and the font you started from was Regular.

10 What Slice does to the rest of the font

You do not have to think about any of this, but it is worth knowing that it happens.

- **Metrics are recalculated** from the resulting outlines: `head`'s bounding box, `hhea`'s extremes, `maxp`'s maxima, and `OS/2.xAvgCharWidth`.
- **`usWeightClass` and `usWidthClass`** follow a pinned `wght` or `wdth` axis, so a font pinned at 700 reports itself as 700.
- `MVAR` — variable vertical metrics — is applied at the location you pinned and then removed, so ascender, descender, x-height and underline land where they should.
- **Variation tables go** when every axis is pinned: `fvar`, `gvar`, `avar`, `HVAR`, `MVAR`, `STAT`'s unreachable values.
- `DSIG`, a digital signature over the font's bytes, is deleted. Instancing rewrites those bytes, so any signature carried over would be invalid, and an invalid signature is worse than none.
- **Feature variations** — the conditional substitutions used by `rvrn` — are resolved at the pinned location, so a font whose italic `a` only appears past a certain slant gets the right `a`.
- **Named instances** that fall outside the new design space are dropped from `fvar`.

11 Known limitations

Stated plainly, because a font tool that is quiet about its gaps is worse than one that has them.

- **CFF 1.0 fonts are refused.** A plain `CFF` font is not variable to begin with. CFF2 variable fonts are supported.
- **`avar` version 2** is refused for partial instancing.
- **`MVAR` across a restricted range** is applied at the new default and then dropped, so vertical metrics are correct there but stop varying across the remaining range.
- **WOFF2 output is about 19% larger** than what `woff2_compress` produces on a glyf-heavy font, because the glyph transform is not implemented on the writing side. The output is a conformant WOFF2 that every browser reads.
- **The engine runs on the main thread**, so the page stops responding while a large font is processed. The progress dialog says so rather than animating a bar that is not measuring anything.

12 Where the evidence is

Every behavioural claim in this manual is tested, and the tests are readable:

- **The test suite in plain English** — all 297 conformance cases, each with the reasoning for why that is the right answer.
- **Adjudication** — every case the original Slice fails, with the measurement behind each verdict.
- **The real-world sweep** — what happens on 775 real variable fonts from Google Fonts: 177,154 glyphs compared against fontTools with no disagreements.
- **The behaviour map** — the numbered map of the original program's behaviour that the suite is written against.

Absolute URLs rather than filenames, so the same line works in the PDF, on the website and in the repository.

13 Credits and licence

Slice is a reimplementation of Slice by Source Foundry, which is the program this one is measured against throughout. The font engine is built on the fontations crates; overlap removal uses linesweeper and kurbo; the interface is Leptos.

See `LICENSE` in the repository for licence terms.